

029 练习详解

从手算导数到框架语义的完整核对

1 仿射层与错误均值

$XW = X$, 每行加 $b = (1/2, -1/2)$, 所以 Z 两行分别为 $(3/2, 3/2)$ 和 $(-1/2, 1/2)$ 。平方和为 $9/4 + 9/4 + 1/4 + 1/4 = 5$ 。正确损失为 $5/(2 \cdot 2) = 5/4$ 。

由 $G = Z/n$ 得到两行 $(3/4, 3/4)$ 和 $(-1/4, 1/4)$ 。 $dW_{11} = 1(3/4) + (-1)(-1/4) = 1$,
 $dW_{12} = 1(3/4) + (-1)(1/4) = 1/2$ 。第二行分别为 $2(3/4) + 1(-1/4) = 5/4$,
 $2(3/4) + 1(1/4) = 7/4$ 。故

$$dW = \begin{bmatrix} 1 & 1/2 \\ 5/4 & 7/4 \end{bmatrix}, \quad db = (1/2, 1), \quad dX = G.$$

`0.5*z.square().mean()`以四个张量元素为均值分母, 损失变为 $5/8$; dZ, dW, db, dX 全部变为上述的一半。偏置在两条样本之间共享, 因此 $\partial L/\partial b$ 是两条路径之和。不能在 G 中已除 n 后再次按样本平均。

2 相同数值为何导数不同

令 $h = w^2$ 且 $w = 2$ 。直接 $h*w$ 与 $h.clone()*w$ 都得到8。`clone`的局部雅可比是恒等映射, 完整乘积法则仍给出 $2w \cdot w + w^2 = 12$ 。`clone`创建新非叶节点和独立存储; 它不是新的独立参数。

`h.detach()*w`也得到8, 但 h 这条路径被挡住, 只有最后一次乘法贡献 $h = 4$ 。分离节点不需要梯度。`torch.tensor(h)*w`同样切断旧历史且复制数值, 因此对原 w 仍得到4。

若写`new_h=torch.tensor(h, requires_grad=True)`, 则`new_h`是一个新叶节点。此时原 w 梯度为4, 新叶`new_h`的梯度为 $w = 2$; “给新张量开启梯度”没有补回`new_h`与 w^2 之间的旧边。框架可能提示应使用显式的`clone/detach`形式, 提示本身也应被认真处理。

只有整个loss就是重建后不需要梯度的张量时, `backward`才因没有求导目标而立即报错。若下游还需要梯度的参数, `loss`依然需要梯度, 断图会以部分参数`None`的方式隐藏。

3 三个上游种子

在 $w = (1, 2)$, $J_u = \text{diag}(2, 4)$ 。种子 $(1, 1)$ 给 $(2, 4)$, 对应 $w_1^2 + w_2^2$; 种子 $(1/2, 1/2)$ 给 $(1, 2)$, 对应两项的均值; 种子 $(3, -1)$ 给 $(6, -4)$, 对应 $3w_1^2 - w_2^2$ 。

这里 u 和`seed`均为形状 $(2,)$ 。不提供`seed`时, 多元素输出并没有唯一的标量目标, 框架不会替你在和、均值或其他组合中选一个。`autograd.grad`返回一个元组, 因为`inputs`可以有多个; $(g,)$ 表示本例只有一个输入对象。它默认不累加到`w.grad`, 所以返回正确的 g 而`w.grad is None`并不矛盾。

若`seed`依赖 w , 完整标量目标的微分是 $d(v^T u) = v^T du + u^T dv$ 。仅把一个当前`seed`送给VJP, 不能忽略它本身在原目标中的变化。最清楚的办法是把预期标量目标明确构造为`tensor`表达式再求导。

4 累加图示与微批量权重

$L(w) = w^2$ 在2的导数始终为4。每次backward得到一份4，缓冲区初始为空、第一次变4、第二次变8。`w.grad=None`清空缓冲区，下一次又是4。若改用`zero_()`，在本例普通累加中也会得到4；在有动量或权重衰减等优化器语义下，“无梯度”和“零梯度”可能不同，本讲没有把两者宣称为普遍等价。

`retain_graph=True`保留用于再次反向的图缓存，不重置缓冲区。两次对同一平方loss反向且保留图，也会累加为8。`retain_grad()`针对中间节点的梯度保存，是第三种不同功能。先看到错误就添加一个`retain`选项，会掩盖真正的寿命管理问题。

设微批A有2条、B有3条。 $L_A = (\ell_1 + \ell_2)/2$ ， $L_B = (\ell_3 + \ell_4 + \ell_5)/3$ 。要恢复 $L = (\ell_1 + \dots + \ell_5)/5$ ，比例只能是2/5和3/5。错误的 $\frac{1}{2}(L_A + L_B)$ 对前两条每条加权1/4、后三条每条1/6。梯度差来自目标不同，不来自PyTorch不会处理小批次。

固定实验正确加权的最大差仅为浮点舍入，错误等权最大差约0.0330774。由于网络无随机模块、无跨样本统计，且所有梯度在同一参数状态下计算，这个分拆比较成立。若在A后更新参数，B的导数点已变；若加入BatchNorm等跨样本操作，前向函数也可能改变，必须重做等价性论证。

5 断图诊断的完整证据

在 $H1=\tanh(Z1)$ 之后把H1替换为`H1.detach`， H_1 数值不变，所以 Z_2, H_2, Z_3, L 完全相同。

W_2, b_2, W_3, b_3 仍参与后面的可微运算，因此这些参数的梯度可用。 X, W_1, b_1 通向loss的唯一通路被切断，其梯度为None。原 Z_1 也没有收到这条反向路径。

正确诊断先确认loss需要梯度，再列出期望叶参数是否可达，而不是在loss上停下。保留各隐藏层梯度后，检查断点两侧的`requires_grad`和`grad_fn`；定位到H1重建或分离。比较表记录loss差为0、最后一层权重梯度差为0和第一层缺失，这组结果排除“只是最后一层运算出错”。

修复需恢复原H1作为第二层输入。若需要独立数值副本且保留导数，使用`clone`；若为了记录可导出日志，写入另一个日志变量，不把日志对象送回模型。修复后重跑所有前向和反向坐标，不能只检查不再报错。

None不能随便转成零。None可能是没有反向路径，也可能仅是一个非叶节点未`retain_grad`。真正的零梯度表示一条有效计算路径在该点给出零，仍应由形状正确的零张量表示。诊断时必须结合节点类型、执行历史和目标路径。

6 三种存储修改风险

需要梯度的叶张量在默认模式直接`add_`，通常在修改发生处立即报错。解决办法不是绕过保护；对于准备输入，在建图前处理；对于普通参数更新，在本次backward结束后于`no_grad`内处理，并重新前向。

保存张量被修改的错误在backward检查时出现。例如先保存`tanh`输出`h`以计算平方loss，再`h.add_(1)`，旧前向对应的保存值已经被破坏。框架发现版本变化而拒绝给出不可靠结果。用`new_h=h+1`产生新对象，并明确loss应依赖`h`还是`new_h`；不能一边保留旧loss，一边假定存储修改也会重算旧loss。

detach别名共享存储，所以从别名修改也会影响原值。no_grad只管新运算是否被记录，不会还原旧图需要的值；它不是“所有原地操作都安全”的上下文。

NumPy别名更危险：在本环境的探针中， $t = 2$ 时前向平方为4，随后从NumPy把底层值改成3，反向观察到6而不是4，并没有同样的版本错误。防护是避免前向与反向间任何外部别名写入，用独立复制建立清楚的所有权边界。不要把这个特定探针的静默行为当成可依赖的跨版本契约。

7 两个独立开关的四种组合

对输入 $(1, 2)$ 且 $p = 1$ 的Dropout：训练模式、启用梯度时输出 $(0, 0)$ ，输出仍处于可求导流程，输入梯度为 $(0, 0)$ ；评估模式、启用梯度时输出 $(1, 2)$ ，对输出求和的输入梯度为 $(1, 1)$ 。

训练模式配合no_grad时，新计算的输出为 $(0, 0)$ 且不记录新的反向历史。评估模式配合no_grad的具体Dropout实现可能直接返回输入这个已有张量，所以不能仅凭结果的requires_grad就断言是否记录了新运算。判断no_grad应另选确实发生新计算的表达式，如`module(x)*2`：在两种模块模式下这个乘法结果都不需要梯度。

输入本身原有requires_grad不会被no_grad永久关闭，旧grad也不会自动清空。工厂函数若显式指定requires_grad=True，还可以在no_grad中创建一个需要梯度的新叶。冻结参数则是明确改变哪些叶需要梯度的设计选择；仅仅不记录一次评估，通常并不改变训练参数的长期状态。

8 高阶导数与证据限制

$f'(w) = (w^2 + w)(2w + 1)$ 。在 $w = -1$ ， $w^2 + w = 0$ ，故一阶为0。对乘积再求导：

$$f''(w) = (2w + 1)^2 + 2(w^2 + w) = 6w^2 + 6w + 1.$$

代入 -1 得1，代入 $1/2$ 得 $11/2$ 。一阶在一点为零不意味着函数在附近是常数，也不意味着二阶为零。

`create_graph=True`记录求导过程，令一阶结果继续可微；`retain_graph=True`保留已有图的数据。保存一阶数值数组后再包装成tensor，只会得到一个新变量，不会自动找回高阶历史。

实验的ReLU扩展也可独立核对：在 $x = 0$ ，框架采用局部反传值0；中心差分

$[\text{ReLU}(h) - \text{ReLU}(-h)]/(2h) = 1/2$ ($h > 0$)。左右导数分别为0和1，所以普通导数不存在；0与 $1/2$ 的差别不能按光滑点标准判为实现错误。

成功的梯度检查不能证明标签或目标定义正确，不能证明训练收敛，也不能证明泛化、无泄漏或现实任务有效。小范围float64验证也不能保证极端输入、其他dtype、GPU或所有版本逐位一致。一个可靠迁移记录应同时保存数据与参数、模型与reduction、软件版本、检查范围、错误探针和未测试限制，而不是只保存一行“allclose通过”。